

Diseño Conceptual y Lógico

ECOMMIFY

Arquitectura Híbrida · PostgreSQL + MongoDB

Etapla 2 — Entregable 1: Diseño Conceptual y Lógico de la Base de Datos

Unidad 2: PostgreSQL — Diseño Relacional Avanzado

MATERIA	ENTREGABLE
Diseño y Optimización de Bases de Datos	Etapla 2 — Evaluativa Mayo 2026

INTEGRANTES DEL GRUPO	
1	Juan José Vélez Álvarez
2	Juan Sebastián Buitrago Romero
3	Sergio Andrés Cobos Suárez
4	David Aníbal Vásquez Beltrán

Indice

<u>1. Portada y Resumen Ejecutivo</u>	<u>4</u>
<u>1.1 Enfoque arquitectónico seleccionado</u>	<u>4</u>
<u>1.2 Abstract</u>	<u>4</u>
<u>2. Análisis de requisitos</u>	<u>5</u>
<u>2.2 Requisitos Funcionales</u>	<u>5</u>
<u>2.3 Requisitos No Funcionales</u>	<u>5</u>
<u>2.4 Identificación de Entidades del Dominio</u>	<u>6</u>
<u>2.5 Tabla de Volúmenes de Datos Esperados</u>	<u>12</u>
<u>3. Diseño conceptual</u>	<u>13</u>
<u>3.1 Diagrama ER</u>	<u>13</u>
<u>3.2 Descripción detallada de cada entidad y relación</u>	<u>14</u>
<u>3.3 Restricciones de Negocio Identificadas</u>	<u>15</u>
<u>3.3.1 Restricciones de integridad referencial</u>	<u>15</u>
<u>3.3.2 Restricciones de dominio (valores válidos)</u>	<u>16</u>
<u>3.3.3 Restricciones de reglas de negocio</u>	<u>18</u>
<u>3.3.4 Restricciones operacionales y de plataforma</u>	<u>20</u>
<u>3.3.5 Restricciones de privacidad y datos sensibles</u>	<u>20</u>
<u>3.3.6 Resumen de restricciones por tipo</u>	<u>21</u>
<u>4. Diseño lógico - Módulo PostgreSQL</u>	<u>22</u>
<u>Esquema relacional normalizado</u>	<u>22</u>
<u>Tabla por tabla con columnas, tipos de datos, claves, constraints</u>	<u>22</u>
<u>Aplicación de tipos avanzados (JSONB, arrays, composite types) justificada</u>	<u>22</u>
<u>Extensiones a utilizar con justificación</u>	<u>22</u>
<u>Estrategia de particionamiento si aplica</u>	<u>22</u>
<u>5. Diseño lógico preliminar- Módulo MongoDB</u>	<u>22</u>
<u>5.1 Colecciones principales identificadas</u>	<u>23</u>
<u>5.2 Esquema de documentos (ejemplo en JSON)</u>	<u>25</u>
<u>5.3 Patrones de modelado a aplicar</u>	<u>29</u>

5.4 Justificación de qué datos van a MongoDB vs PostgreSQL	32
6. Decisiones arquitectónicas justificadas	32
<u>Matriz de decisión por entidad</u>	33
<u>Análisis del teorema CAP aplicado</u>	33
<u>Análisis de Trade-offs y Estrategias de Mitigación</u>	33
<u>Flujos de sincronización si aplica</u>	35
7. Anexos	36
7.1 Diccionario de Datos Ecommify	36
7.2 Scripts SQL preliminares	39
7.3 Ejemplos de datos	39

1. Portada y Resumen Ejecutivo

1.1 Enfoque arquitectónico seleccionado

El equipo ha adoptado una arquitectura políglota híbrida que combina PostgreSQL como motor relacional para el módulo transaccional y MongoDB Atlas como base de datos documental para el módulo de catálogo y reseñas. Esta decisión responde directamente a la naturaleza dual del dataset de Ecommify (Brazilian E-Commerce de Olist): un núcleo transaccional altamente estructurado con integridad referencial completa, y un catálogo de productos con esquema variable y texto libre que se adapta mejor a un modelo documental.

PostgreSQL	MongoDB
Módulo transaccional	Módulo documental
• Customers, Orders, Order_Items • Order_Payments, Sellers • Geolocation (deduplicada) • Integridad referencial ACID	• Products + Category (embebida) • Order_Reviews (texto libre) • Esquema flexible por categoría • Alta disponibilidad BASE

1.2 Abstract

Resumen ejecutivo — Decisiones clave del diseño

El presente entregable documenta el diseño conceptual y lógico de la base de datos del proyecto Ecommify, una plataforma de comercio electrónico basada en el dataset Brazilian E-Commerce de Olist. El equipo adoptó una arquitectura híbrida que distribuye las nueve entidades del dominio entre dos sistemas de gestión de bases de datos según la naturaleza intrínseca de cada dato. PostgreSQL, desplegado en Supabase, aloja el módulo transaccional compuesto por Customers, Orders, Order_Items, Order_Payments y Sellers, entidades que exhiben integridad referencial completa — validada con cero registros huérfanos en el EDA— y patrones de acceso OLTP que exigen propiedades ACID. MongoDB Atlas, en su tier gratuito M0, gestiona Products y Order_Reviews: el catálogo presenta 73 categorías con atributos heterogéneos que no comparten esquema fijo, mientras que las reseñas combinan una calificación numérica estructurada con texto libre opcional con un 58.7% de nulos, características que favorecen el modelo documental. La normalización alcanza mínimo 3FN en el módulo relacional, eliminando las dependencias parciales y transitivas identificadas en la tabla desnormalizada orders_full. Se aplican tipos avanzados de PostgreSQL —JSONB, arrays y rangos de

fecha— en campos donde la estructura variable lo justifica. La frontera entre sistemas está definida por criterios de consistencia, estructura del dato y patrón de consulta, alineados con el Teorema CAP.

El diseño garantiza que la clave compartida `product_id` mantenga consistencia entre ambos sistemas mediante scripts de reconciliación ejecutados en Google Colab, compensando la ausencia de Change Streams en el tier gratuito de MongoDB Atlas. Las decisiones arquitectónicas priorizan la escalabilidad, la integridad de datos transaccionales y la flexibilidad del catálogo, dentro de las limitaciones técnicas de las plataformas gratuitas seleccionadas para el proyecto académico.

2. Análisis de requisitos

2.1 Contexto

Ecommify es una plataforma basada en el ecosistema de comercio electrónico que gestiona el ciclo de vida completo de compras en línea. Su base de datos abarca nueve áreas de información principal, integrando clientes, pedidos, productos, pagos, reseñas, vendedores y geolocalización. El proyecto busca estructurar los datos transaccionales mediante un esquema fuertemente normalizado para un entorno OLTP, garantizando la consistencia de los procesos del negocio y mitigando anomalías, lo que a su vez sienta las bases para futuras integraciones en arquitecturas políglotas o analíticas.

2.2 Requisitos Funcionales

- **Gestión de Órdenes:** El sistema debe registrar y administrar todo el ciclo de vida de los pedidos, controlando estados y las múltiples fechas asociadas a la aprobación, envío y entrega.
- **Gestión de Entidades Comerciales:** Debe permitir el almacenamiento de clientes y vendedores (Sellers), vinculándolos a un sistema de geolocalización a través de sus respectivos prefijos de códigos postales.
- **Catálogo de Productos:** Administrar un inventario heterogéneo de productos organizados por categorías, soportando atributos de distinta granularidad y traducciones de nombres de categoría (por ejemplo, al inglés).
- **Transacciones Financieras:** Registrar los diferentes métodos de pago utilizados en las compras y dar soporte a escenarios secuenciales, es decir, donde un solo pedido requiere de múltiples pagos.
- **Retroalimentación del Usuario:** El sistema debe permitir almacenar y vincular las reseñas o calificaciones generadas por los usuarios con sus respectivos pedidos.

2.3 Requisitos No Funcionales

- **Integridad Referencial y Consistencia:** La arquitectura transaccional debe estar normalizada como mínimo hasta la Tercera Forma Normal (3FN) para evitar duplicidad masiva de información en cabeceras y eliminar dependencias transitivas.

- Escalabilidad de Lectura (OLAP): El diseño de la base de datos debe contemplar estrategias de desnormalización, como tablas de agregación o vistas materializadas, para no comprometer el rendimiento general al ejecutar reportes de análisis de negocio.
- Rendimiento en Altos Volúmenes: El sistema debe poder manejar eficientemente tablas estructurales de alto peso (como la de geolocalización, que supera el millón de registros) sin degradar las transacciones de compra.

2.4 Identificación de Entidades del Dominio

Basado en el diseño del esquema relacional normalizado para el negocio, se identifican formalmente 9 entidades principales:

Tabla: Customers

- Propósito: Contiene información sobre los clientes.
- Clave Primaria (PK): customer_id
- Relaciones: customer_zip_code_prefix (FK a Geolocation)

Columna	Tipo de Dato	Descripción	Clave	Restricciones / Notas
 customer_id	TEXT	Identificador único del cliente.	PK	• No nulo • Clave primaria de la tabla
 customer_unique_id	TEXT	Identificador único del cliente (para evitar duplicidad).	—	• No nulo • Debe ser único (UNIQUE)
 customer_zip_code_prefix	INT	Prefijo del código postal del cliente.	FK	• No nulo • Clave foránea hacia Geolocation(customer_zip_code_prefix)
 customer_city	TEXT	Ciudad de residencia del cliente.	—	• No nulo • Almacena el nombre de la ciudad
 customer_state	TEXT	Estado o provincia de residencia del cliente.	—	• No nulo • Código o nombre del estado (ej. SP)
PK Clave Primaria FK Clave Foránea TEXT Tipo de dato de texto INT Tipo de dato entero				

Tabla: Geolocation

- Propósito: Contiene datos de geolocalización para prefijos de códigos postales.
- Clave Primaria (PK): geolocation_zip_code_prefix
- Relaciones: No tiene FKs entrantes en este contexto, pero es referenciada por Customers y Sellers.

Columna	Tipo de Dato	Descripción	Clave	Restricciones / Notas
 geolocation_zip_code_prefix	INT	Prefijo del código postal.	PK	• No nulo • Clave primaria de la tabla • Formato numérico (ej. 01001)
 geolocation_lat	FLOAT	Latitud asociada al código postal.	—	• No nulo • Valores entre -90 y 90
 geolocation_lng	FLOAT	Longitud asociada al código postal.	—	• No nulo • Valores entre -180 y 180
 geolocation_city	TEXT	Ciudad asociada al código postal.	—	• No nulo • Nombre oficial de la ciudad
 geolocation_state	TEXT	Estado o provincia asociado al código postal.	—	• No nulo • Código o nombre del estado (ej. SP, São Paulo)

PK

Clave Primaria

FK

Clave Foránea

TEXT

Tipo de dato de texto

INT

Tipo de dato entero

FLOAT

Tipo de dato decimal flotante

Tabla: Orders

- Propósito: Contiene información a nivel de pedido.
- Clave Primaria (PK): order_id
- Relaciones: customer_id (FK a Customers)

Columna	Tipo de Dato	Descripción	Clave	Restricciones / Notas
 order_id	TEXT	Identificador único del pedido.	PK	• No nulo • Clave primaria de la tabla
 customer_id	TEXT	Identificador del cliente que realiza el pedido.	FK	• No nulo • Clave foránea hacia Customers(customer_id)
 order_status	TEXT	Estado actual del pedido.	—	• No nulo • Valores controlados (ej. delivered, shipped, canceled, etc.)
 order_purchase_timestamp	DATETIME	Fecha y hora en que se realizó la compra.	—	• No nulo • Fecha y hora de creación del pedido
 order_approved_at	DATETIME	Fecha y hora de aprobación del pago.	—	• Puede ser nulo • Nulo si el pago aún no ha sido aprobado
 order_delivered_carrier_date	DATETIME	Fecha y hora de entrega al transportista.	—	• Puede ser nulo • Indica cuándo el pedido fue entregado al transportista
 order_delivered_customer_date	DATETIME	Fecha y hora de entrega al cliente.	—	• Puede ser nulo • Indica cuándo el cliente recibió el pedido
 order_estimated_delivery_date	DATETIME	Fecha estimada de entrega al cliente.	—	• Puede ser nulo • Fecha estimada calculada al momento de la compra
PK Clave Primaria FK Clave Foránea TEXT Tipo de dato de texto DATETIME Tipo de dato fecha y hora				

Tabla: Products

- Propósito: Contiene información detallada sobre los productos.
- Clave Primaria (PK): product_id
- Relaciones: product_category_name (FK a Product_Category_Name_Translation)

Columna	Tipo de Dato	Descripción	Clave	Restricciones / Notas
 product_id	TEXT	Identificador único del producto.	PK	• No nulo • Clave primaria de la tabla
 product_category_name	TEXT	Nombre de la categoría del producto.	FK	• No nulo • Clave foránea hacia Product_Category(product_category_name)
 product_name_lenght	INT	Longitud del nombre del producto.	—	• Puede ser nulo • Cantidad de caracteres del nombre
 product_description_lenght	INT	Longitud de la descripción del producto.	—	• Puede ser nulo • Cantidad de caracteres de la descripción
 product_photos_qty	INT	Cantidad de fotos del producto.	—	• Puede ser nulo • Número total de imágenes del producto
 product_weight_g	FLOAT	Peso del producto en gramos.	—	• Puede ser nulo • Mayor o igual a 0
 product_length_cm	FLOAT	Longitud del producto en centímetros.	—	• Puede ser nulo • Mayor o igual a 0
 product_height_cm	FLOAT	Altura del producto en centímetros.	—	• Puede ser nulo • Mayor o igual a 0
 product_width_cm	FLOAT	Ancho del producto en centímetros.	—	• Puede ser nulo • Mayor o igual a 0

PK

 Clave Primaria

FK

 Clave Foránea

TEXT

 Tipo de dato de texto

INT

 Tipo de dato entero

FLOAT

 Tipo de dato decimal flotante

Tabla: Sellers

- Propósito: Contiene información sobre los vendedores.
- Clave Primaria (PK): seller_id
- Relaciones: seller_zip_code_prefix (FK a Geolocation)

Columna	Tipo de Dato	Descripción	Clave	Restricciones / Notas
 seller_id	TEXT	Identificador único del vendedor.	PK	• No nulo • Clave primaria de la tabla
 seller_zip_code_prefix	INT	Prefijo del código postal del vendedor.	FK	• No nulo • Clave foránea hacia Geolocation(seller_zip_code_prefix) • Formato numérico (ej. 01001)
 seller_city	TEXT	Ciudad de residencia del vendedor.	—	• No nulo • Almacena el nombre de la ciudad
 seller_state	TEXT	Estado o provincia de residencia del vendedor.	—	• No nulo • Código o nombre del estado (ej. SP, São Paulo)

PK

 Clave Primaria

FK

 Clave Foránea

TEXT

 Tipo de dato de texto

INT

 Tipo de dato entero

Tabla: Order_Items

- Propósito: Contiene los detalles de los ítems individuales dentro de cada pedido. Es una tabla de hechos que une Orders, Products y Sellers.
- Clave Primaria (PK): (order_id, order_item_id) (Clave compuesta)
- Relaciones: order_id (FK a Orders), product_id (FK a Products), seller_id (FK a Sellers)

Columna	Tipo de Dato	Descripción	Clave	Restricciones / Notas
 order_id	TEXT	Identificador del pedido (Parte de PK, FK).	PK, FK	• No nulo • Forma parte de la clave primaria compuesta
 order_item_id	INT	Identificador del ítem dentro del pedido (Parte de PK).	PK	• No nulo • Forma parte de la clave primaria compuesta
 product_id	TEXT	Identificador del producto (FK).	FK	• No nulo • Clave foránea hacia Products(product_id)
 seller_id	TEXT	Identificador del vendedor (FK).	FK	• No nulo • Clave foránea hacia Sellers(seller_id)
 shipping_limit_date	DATETIME	Fecha límite para el envío del ítem.	—	• Puede ser nulo • Fecha y hora límite para despachar el ítem
 price	FLOAT	Precio unitario del ítem al momento de la compra.	—	• No nulo • Precio registrado en la compra (snapshot)
 freight_value	FLOAT	Valor del flete asociado al ítem.	—	• No nulo • Valor del envío asignado al ítem (snapshot)

PK

 Clave Primaria

FK

 Clave Foránea

TEXT

 Tipo de dato de texto

INT

 Tipo de dato entero

FLOAT

 Tipo de dato decimal flotante

DATETIME

 Tipo de dato fecha y hora

Tabla: Order_Payments

- Propósito: Contiene los detalles de los pagos asociados a cada pedido.
- Clave Primaria (PK): (order_id, payment_sequential) (Clave compuesta)
- Relaciones: order_id (FK a Orders)

Columna	Tipo de Dato	Descripción	Clave	Restricciones / Notas
 order_id	TEXT	Identificador del pedido (Parte de PK, FK).	PK, FK	• No nulo • Clave foránea hacia Orders(order_id)
 payment_sequential	INT	Secuencia de pago dentro del pedido (Parte de PK).	PK	• No nulo • Inicia en 1 y se incrementa por cada pago registrado del pedido
 payment_type	TEXT	Tipo de pago utilizado.	—	• No nulo • Valores controlados (ej. credit_card, boleto, voucher, debit_card, etc.)
 payment_installments	INT	Número de cuotas del pago.	—	• No nulo • Valor entero positivo (≥ 1)
 payment_value	FLOAT	Valor total del pago registrado.	—	• No nulo • Valor decimal mayor o igual a 0 • Moneda: BRL (Real Brasileño)

PK

 Clave Primaria

FK

 Clave Foránea

TEXT

 Tipo de dato de texto

INT

 Tipo de dato entero

FLOAT

 Tipo de dato decimal flotante

Tabla: Order_Reviews

- Propósito: Contiene las reseñas de los clientes para los pedidos.
- Clave Primaria (PK): review_id
- Relaciones: order_id (FK a Orders)

Columna	Tipo de Dato	Descripción	Clave	Restricciones / Notas
 review_id	TEXT	Identificador único de la reseña (PK).	PK	• No nulo • Clave primaria de la tabla
 order_id	TEXT	Identificador del pedido (FK).	FK	• No nulo • Clave foránea hacia Orders(order_id)
 review_score	INT	Puntuación de la reseña en una escala de 1 a 5.	—	• No nulo • Valores permitidos: 1, 2, 3, 4, 5
 review_comment_title	TEXT	Título del comentario de la reseña.	—	• Puede ser nulo • Longitud máxima recomendada: 150 caracteres
 review_comment_message	TEXT	Mensaje del comentario de la reseña.	—	• Puede ser nulo • Longitud máxima recomendada: 2000 caracteres
 review_creation_date	DATETIME	Fecha de creación de la reseña.	—	• No nulo • Fecha y hora en que se creó la reseña
 review_answer_timestamp	DATETIME	Fecha y hora de respuesta a la reseña.	—	• Puede ser nulo • Fecha y hora en que se respondió la reseña

PK

 Clave Primaria

FK

 Clave Foránea

TEXT

 Tipo de dato de texto

INT

 Tipo de dato entero

DATETIME

 Tipo de dato fecha y hora

Tabla: Product_Category_Name_Translation

- Propósito: Proporciona traducciones de los nombres de categorías de productos del portugués al inglés.
- Clave Primaria (PK): product_category_name
- Relaciones: No tiene FKs, pero es referenciada por Products.

Columna	Tipo de Dato	Descripción	Clave	Restricciones / Notas
 product_category_name	TEXT	Nombre de la categoría en portugués (PK).	PK	• No nulo • Clave primaria de la tabla
 product_category_name_english	TEXT	Nombre de la categoría en inglés.	—	• Puede ser nulo • Traducción de la categoría a inglés

PK

 Clave Primaria

FK

 Clave Foránea

TEXT

 Tipo de dato de texto

INT

 Tipo de dato entero

FLOAT

 Tipo de dato decimal flotante

DATETIME

 Tipo de dato fecha y hora

Este esquema normalizado elimina la redundancia observada en orders_full y prepara los datos para un modelado de base de datos relacional más eficiente y consistente.

2.5 Tabla de Volúmenes de Datos Esperados

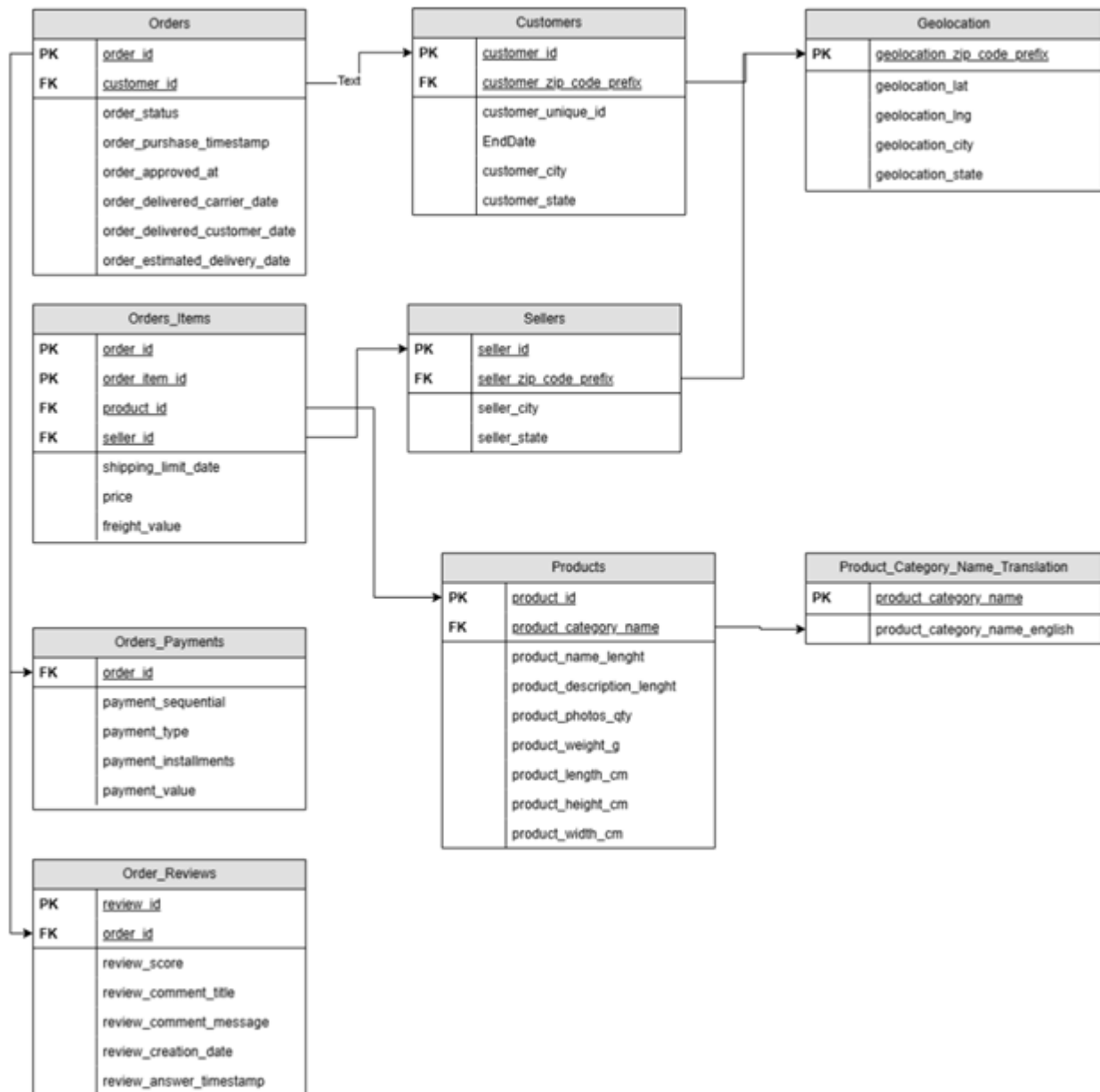
A partir de los hallazgos del primer análisis exploratorio de Ecommify, las expectativas volumétricas de las entidades principales se comportan de la siguiente manera:

● Volúmenes de Datos Esperados Proyecto Ecommify — Arquitectura Transaccional ANÁLISIS DE REQUISITOS		
ENTIDAD / TABLA	VOLUMEN APROXIMADO	JUSTIFICACIÓN OPERATIVA
Geolocation	> 1,000,000 registros	Es la entidad más voluminosa, almacena millones de coordenadas geográficas atadas a códigos postales, y presenta un alto volumen histórico de duplicados.
Customers	~99,441 registros	Corresponde al volumen total en la base, lo cual representa alrededor de 96,096 clientes únicos (revelando una tasa de clientes con órdenes múltiples).
Orders	~99,000+ registros	Alto volumen de compras acumuladas; la gran mayoría de estas (96,478 registros) representan un flujo exitoso en estado "delivered".
Order_Payments	~100,000+ registros	Se espera que el volumen supere las transacciones regulares, dado que solo las compras con tarjeta de crédito agrupan 76,795 registros, existiendo casos de múltiples pagos secuenciales por orden.
Products	32,951 registros	Volumen de artículos únicos heterogéneos activos e identificados dentro del catálogo general.

3. Diseño conceptual

3.1 Diagrama ER

Fase Conceptual - Modelo Entidad Relación



3.2 Descripción detallada de cada entidad y relación

Entidades principales

ENTIDAD	ROL EN EL DOMINIO
Customer	Actor que genera pedidos
Order	Transacción principal del negocio
Order Item	Ítem individual dentro de un pedido
Product	Artículo vendido (catalogo)
Category	Clasificación del producto
Seller	Vendedor responsable de cada ítem
Payment	Forma y valor de pago del pedido
Review	Evaluación post entrega del cliente
Geolocation	Referencia geográfica por código postal

Entidades principales y sus relaciones

Relación y cardinalidad
Customer —(1:N)—► Order
Order —(1:N)—► Order Item
Product —(1:N)—► Order Item
Seller —(1:N)—► Order Item
Category —(1:N)—► Product
Order —(1:N)—► Payment
Order —(1:0..1)— Review Geolocation —(1:N)—► Customer (vía zip_code_prefix)
Geolocation —(1:N)—► Seller (vía zip_code_prefix)

3.3 Restricciones de Negocio Identificadas

Las restricciones de negocio son reglas que el sistema de base de datos debe enforzar para garantizar que los datos reflejen fielmente las reglas del dominio del negocio de e-commerce. Estas restricciones trascienden la normalización técnica y se derivan del análisis del dataset Olist y del comportamiento real del negocio identificado en el EDA.

3.3.1 Restricciones de integridad referencial

Derivadas directamente del modelo relacional y validadas por el EDA, que confirmó cero registros huérfanos en todas las relaciones. Estas restricciones se implementan mediante FOREIGN KEY con las políticas ON DELETE y ON UPDATE apropiadas.

#	Restricción	Tipo	Entidades afectadas	Implementación en BD
R-01	Todo pedido debe pertenecer a un cliente existente	Integridad referencial	Orders → Customers	FK customer_id NOT NULL REFERENCES customers ON DELETE RESTRICT
R-02	Todo ítem debe referenciar un pedido, producto y vendedor válidos	Integridad referencial	Order_Items → Orders, Products, Sellers	FK compuesta con ON DELETE RESTRICT en las 3 referencias
R-03	Todo pago debe estar asociado a un pedido existente	Integridad referencial	Order_Payments → Orders	FK order_id NOT NULL ON DELETE CASCADE
R-04	Toda reseña debe referenciar un pedido válido	Integridad referencial	Order_Reviews → Orders	FK order_id NOT NULL ON DELETE CASCADE
R-05	Todo producto debe tener una categoría registrada en la tabla de traducción	Integridad referencial	Products → Category_Translation	FK product_category_name ON DELETE SET NULL
R-06	Cientes y vendedores deben referenciar un código postal registrado en Geolocation	Integridad referencial	Customers, Sellers → Geolocation	FK zip_code_prefix ON DELETE RESTRICT

3.3.2 Restricciones de dominio (valores válidos)

Reglas sobre los valores permitidos en cada atributo, derivadas del análisis de distribuciones del EDA y las reglas de negocio del e-commerce.

#	Restricción	Tipo	Entidades afectadas	Implementación en BD
R-07	El score de una reseña solo puede ser un entero entre 1 y 5	Domino	Order_Reviews	CHECK (review_score BETWEEN 1 AND 5)
R-08	El estado de un pedido debe ser uno de los valores del ciclo de vida definido	Domino	Orders	CHECK (order_status IN ('created','approved','invoiced','processing','shipped','delivered','canceled','unavailable'))
R-09	El precio de un ítem y el valor de flete no pueden ser negativos	Domino	Order_Items	CHECK (price >= 0) y CHECK (freight_value >= 0)
R-10	El valor de un pago debe ser mayor que cero	Domino	Order_Payments	CHECK (payment_value > 0)

R - 1 1	El número de cuotas debe ser un entero positivo	Domini	Order_Payments	CHECK (payment_installments >= 1)
R - 1 2	El tipo de pago debe ser uno de los métodos aceptados por la plataforma	Domini	Order_Payments	CHECK (payment_type IN ('credit_card','debit_card','voucher','boleto','not_defined'))
R - 1 3	Las dimensiones físicas del producto (peso, largo, alto, ancho) no pueden ser negativas si están presentes	Domini	Products	CHECK con condición: columna IS NULL OR columna >= 0

3.3.3 Restricciones de reglas de negocio

Reglas derivadas del funcionamiento del negocio de marketplace que deben ser garantizadas por la base de datos, identificadas mediante el análisis del flujo de vida de una transacción en Ecommify.

#	Restricción	Tipo	Entidades afectadas	Implementación en BD
R-14	La fecha de aprobación del pedido no puede ser anterior a la fecha de compra	Negocio	Orders	CHECK (order_approved_at IS NULL OR order_approved_at >= order_purchase_timestamp)
R-15	La fecha de entrega real no puede ser anterior a la fecha de despacho al transportista	Negocio	Orders	CHECK (order_delivered_customer_date IS NULL OR order_delivered_customer_date >= order_delivered_carrier_date)
R-16	La fecha límite de envío del vendedor no puede ser anterior a la fecha de compra	Negocio	Order_Items	CHECK (shipping_limit_date >= (SELECT order_purchase_timestamp FROM orders WHERE order_id = order_items.order_id))
R-17	Un pedido no puede tener payment_sequential duplicado (no se puede repetir la misma secuencia de pago en el mismo pedido)	Negocio	Order_Payments	UNIQUE (order_id, payment_sequential) — ya parte de la PK compuesta
R-18	Un pedido no puede tener más de una reseña por cliente (1:1 entre pedido y reseña según el modelo de negocio)	Negocio	Order_Reviews	UNIQUE (order_id) — o gestión por lógica de aplicación
R-19	El customer_unique_id identifica al cliente real a través de múltiples	Negocio	Customers	customer_unique_id TEXT NOT NULL

	sesiones; no puede ser nulo			
--	-----------------------------	--	--	--

3.3.4 Restricciones operacionales y de plataforma

Restricciones derivadas de las limitaciones técnicas de las plataformas gratuitas seleccionadas (Supabase Free Tier y MongoDB Atlas M0) y que impactan las decisiones de diseño físico.

#	Restricción	Tipo	Entidades afectadas	Implementación en BD
R-20	La tabla Geolocation debe ser deduplicada antes de cargar para no superar los 500 MB de Supabase Free Tier	Operacional	Geolocation	Estrategia ETL: deduplicar en Colab antes de INSERT. 1M filas → ~150K únicas
R-21	Las colecciones MongoDB deben mantenerse por debajo de 512 MB (límite Atlas M0)	Operacional	Products, Order_Reviews	Monitoreo post-carga: Products (32.951 docs) + Reviews (99.224 docs) ≈ 80 MB
R-22	La sincronización entre PostgreSQL y MongoDB no puede ser en tiempo real (no hay Change Streams en M0)	Operacional	Products (clave compartida product_id)	Script de reconciliación manual en Colab para cruzar product_ids entre sistemas
R-23	Todos los campos NOT NULL deben definirse explícitamente para campos críticos del modelo transaccional	Operacional	Todas las tablas PostgreSQL	order_id, customer_id, product_id, seller_id definidos como NOT NULL en el DDL

3.3.5 Restricciones de privacidad y datos sensibles

Restricciones relacionadas con la protección de datos personales de clientes y la anonimización observada en el dataset original.

#	Restricción	Tipo	Entidades afectadas	Implementación en BD
R-24	Los identificadores de clientes en el dataset están hashed (customer_id y customer_unique_id son UUIDs anonimizados); el diseño debe preservar esta anonimización	Seguridad	Customers	Los IDs se almacenan como TEXT sin revertir el hash. No se almacenan nombres ni emails reales
R-25	Los identificadores de vendedores también están anonimizados en el dataset original	Seguridad	Sellers	seller_id almacenado como TEXT (UUID hasheado). No exposición de datos comerciales reales
R-26	Los review_id son únicos y no deben permitir duplicados que puedan inflar artificialmente puntuaciones	Seguridad	Order_Reviews	review_id TEXT PRIMARY KEY con constraint UNIQUE garantizado por la PK

3.3.6 Resumen de restricciones por tipo

Tipo de restricción	Cantidad	Implementación principal
Integridad referencial	6 (R-01 a R-06)	FOREIGN KEY + ON DELETE RESTRICT/CASCADE
Dominio	7 (R-07 a R-13)	CHECK constraints sobre columnas individuales
Negocio	6 (R-14 a R-19)	CHECK entre columnas + UNIQUE constraints

Operacional	4 (R-20 a R-23)	ETL en Colab + monitoreo externo
Seguridad	3 (R-24 a R-26)	Diseño del modelo + PRIMARY KEY
TOTAL	26 restricciones	5 categorías cubriendo integridad, dominio, negocio, operación y seguridad

Nota de implementación

Las restricciones de integridad referencial (R-01 a R-06) y de dominio (R-07 a R-13) se implementan directamente en el DDL de PostgreSQL mediante FOREIGN KEY y CHECK constraints. Las restricciones de negocio (R-14 a R-19) se implementan mediante CHECK constraints que pueden requerir subqueries, complementadas con triggers cuando la lógica involucra múltiples tablas. Las restricciones operacionales (R-20 a R-23) se gestionan a nivel de ETL en Colab antes de la carga de datos. Las restricciones de privacidad (R-24 a R-26) están garantizadas por el diseño del modelo de datos.

4. Diseño lógico - Módulo PostgreSQL

El módulo PostgreSQL implementa el núcleo transaccional (OLTP) de Ecommify. Su diseño lógico parte del modelo conceptual de la Sección 3 y de las 26 restricciones (R-01 a R-26) ya identificadas, traduciéndolas a un esquema normalizado, tipado y con integridad declarativa. Todo el DDL presentado en esta sección fue validado contra PostgreSQL 16: los scripts se ejecutan sin errores y la reconciliación de la clave compartida `product_id` devuelve cero registros huérfanos.

4.1 Esquema relacional normalizado (3FN)

La normalización elimina las anomalías de inserción, actualización y borrado que presenta la tabla desnormalizada `orders_full` identificada en el EDA. El módulo transaccional alcanza como mínimo la Tercera Forma Normal (3FN); a continuación se documenta el proceso forma por forma con ejemplos concretos del dominio Olist.

Primera Forma Normal (1FN) — atomicidad y eliminación de grupos repetitivos.

Un mismo pedido contiene varios productos. Mantenerlos como grupo repetitivo dentro de `orders_full` (`producto_1`, `producto_2`, ...) viola la 1FN. Se extrae la entidad `Order_Items`, donde cada ítem es una fila atómica identificada por (`order_id`, `order_item_id`). Del mismo modo, los pagos múltiples de un pedido se aíslan en `Order_Payments`.

Segunda Forma Normal (2FN) — eliminación de dependencias parciales.

En `Order_Items` la clave primaria es compuesta (`order_id`, `order_item_id`). Atributos como el peso, las dimensiones o la categoría del producto no dependen de la línea de pedido sino

únicamente de product_id; mantenerlos allí sería una dependencia parcial. Se trasladan a la entidad Products. Igualmente, price y freight_value sí dependen de la clave completa y permanecen en Order_Items.

Tercera Forma Normal (3FN) — eliminación de dependencias transitivas.

En Customers, los atributos customer_city y customer_state no dependen de customer_id sino del prefijo postal (customer_id → zip → ciudad/estado): una dependencia transitiva. Se resuelve extrayendo la entidad Geolocation; Customers y Sellers conservan solo el prefijo como clave foránea. De forma análoga, product_category_name_english depende de product_category_name y no de product_id, por lo que se aísla en la tabla Product_Category_Name_Translation.

Esquema relacional resultante (9 relaciones en 3FN).

```
Geolocation(geolocation_zip_code_prefix PK, lat, lng, city, state)
Customers(customer_id PK, customer_unique_id, customer_zip_code_prefix FK->Geolocation)
Sellers(seller_id PK, seller_zip_code_prefix FK->Geolocation)
Product_Category_Name_Translation(product_category_name PK, product_category_name_english)
Products(product_id PK, product_category_name FK->Translation, name_length,
description_length, photos_qty, weight_g, length_cm, height_cm, width_cm)
Orders(order_id PK, customer_id FK->Customers, order_status, order_purchase_timestamp,
order_approved_at, order_delivered_carrier_date,
order_delivered_customer_date, order_estimated_delivery_date)
Order_Items(order_id FK->Orders, order_item_id, product_id FK->Products,
seller_id FK->Sellers, shipping_limit_date, price, freight_value,
PK(order_id, order_item_id))
Order_Payments(order_id FK->Orders, payment_sequential, payment_type,
payment_installments, payment_value, PK(order_id, payment_sequential))
Order_Reviews(review_id PK, order_id FK->Orders UNIQUE, review_score,
review_comment_title, review_comment_message,
review_creation_date, review_answer_timestamp)
```

Resumen de la normalización

Forma	Anomalía corregida	Decisión de diseño
1FN	Grupos repetitivos de productos/pagos en el pedido	Entidades Order_Items y Order_Payments con filas atómicas
2FN	Atributos de producto dependientes solo de product_id	Atributos movidos a Products; PK compuesta solo retiene lo dependiente de la clave completa
3FN	ciudad/estado dependientes del zip; traducción dependiente de la categoría	Entidades Geolocation y Product_Category_Name_Translation

4.2 Tabla por tabla: columnas, tipos, claves y constraints

Se documenta cada relación con sus columnas, tipos de datos, claves y constraints. Las referencias R-xx remiten a las restricciones de la Sección 3.3. Los identificadores del dataset

son UUID hasheados, por lo que se modelan como VARCHAR(32) preservando la anonimización (R-24, R-25).

Geolocation

Columna	Tipo	Clave / Constraint	Descripción
geolocation_zip_code_prefix	INTEGER	PK	Prefijo postal (deduplicado en ETL, R-20)
geolocation_lat	DOUBLE PRECISION	—	Latitud
geolocation_lng	DOUBLE PRECISION	—	Longitud
geolocation_city	VARCHAR(120)	—	Ciudad
geolocation_state	CHAR(2)	CHECK (~ '^ [A-Z] {2}\$')	Unidad federativa (UF)

Customers

Columna	Tipo	Clave / Constraint	Descripción
customer_id	VARCHAR(32)	PK	Identificador por pedido
customer_unique_id	VARCHAR(32)	NOT NULL (R-19)	Cliente real entre sesiones
customer_zip_code_prefix	INTEGER	FK->Geolocation, ON DELETE RESTRICT (R-06)	Prefijo postal del cliente

Sellers

Columna	Tipo	Clave / Constraint	Descripción
seller_id	VARCHAR(32)	PK	Identificador del vendedor
seller_zip_code_prefix	INTEGER	FK->Geolocation, ON DELETE RESTRICT (R-06)	Prefijo postal del vendedor

Product_Category_Name_Translation

Columna	Tipo	Clave / Constraint	Descripción
product_category_name	VARCHAR(80)	PK	Nombre de categoría (PT)
product_category_name_english	VARCHAR(80)	NOT NULL	Nombre de categoría (EN)

Products

Columna	Tipo	Clave / Constraint	Descripción
product_id	VARCHAR(32)	PK	Identificador de producto (clave compartida con MongoDB)
product_category_name	VARCHAR(80)	FK->Translation, ON DELETE SET NULL (R-05)	Categoría

product_name_length	INTEGER	CHECK (>=0)	Longitud del nombre
product_description_length	INTEGER	CHECK (>=0)	Longitud de la descripción
product_photos_qty	INTEGER	CHECK (>=0)	Cantidad de fotos
product_weight_g	INTEGER	CHECK (IS NULL OR >=0) (R-13)	Peso en gramos
product_length_cm	INTEGER	CHECK (IS NULL OR >=0) (R-13)	Largo
product_height_cm	INTEGER	CHECK (IS NULL OR >=0) (R-13)	Alto
product_width_cm	INTEGER	CHECK (IS NULL OR >=0) (R-13)	Ancho

Orders

Columna	Tipo	Clave / Constraint	Descripción
order_id	VARCHAR(32)	PK	Identificador del pedido (clave compartida)
customer_id	VARCHAR(32)	FK->Customers NOT NULL, ON DELETE RESTRICT (R-01)	Cliente del pedido
order_status	VARCHAR(20)	NOT NULL, CHECK IN (...) (R-08)	Estado del ciclo de vida
order_purchase_timestamp	TIMESTAMP	NOT NULL	Fecha de compra
order_approved_at	TIMESTAMP	CHECK (>= purchase) (R-14)	Fecha de aprobación
order_delivered_carrier_date	TIMESTAMP	—	Despacho al transportista
order_delivered_customer_date	TIMESTAMP	CHECK (>= carrier) (R-15)	Entrega al cliente
order_estimated_delivery_date	TIMESTAMP	—	Entrega estimada

Order_Items

Columna	Tipo	Clave / Constraint	Descripción
order_id	VARCHAR(32)	PK, FK->Orders, ON DELETE RESTRICT (R-02)	Pedido
order_item_id	INTEGER	PK	Secuencia del ítem dentro del pedido
product_id	VARCHAR(32)	FK->Products NOT NULL (R-02)	Producto
seller_id	VARCHAR(32)	FK->Sellers NOT NULL (R-02)	Vendedor

shipping_limit_date	TIMESTAMP	trigger (>= purchase) (R-16)	Límite de envío del vendedor
price	NUMERIC(12,2)	CHECK (>=0) (R-09)	Precio del ítem
freight_value	NUMERIC(12,2)	CHECK (>=0) (R-09)	Valor del flete

Order_Payments

Columna	Tipo	Clave / Constraint	Descripción
order_id	VARCHAR(32)	PK, FK->Orders, ON DELETE CASCADE (R-03)	Pedido
payment_sequential	INTEGER	PK (R-17)	Secuencia del pago
payment_type	VARCHAR(20)	CHECK IN (...) (R-12)	Método de pago
payment_installments	INTEGER	CHECK (>=1) (R-11)	Número de cuotas
payment_value	NUMERIC(12,2)	CHECK (>0) (R-10)	Valor del pago

Order_Reviews

Columna	Tipo	Clave / Constraint	Descripción
review_id	VARCHAR(32)	PK (R-26)	Identificador de la reseña
order_id	VARCHAR(32)	FK->Orders NOT NULL, UNIQUE (R-04, R-18), ON DELETE CASCADE	Pedido reseñado (1:1)
review_score	SMALLINT	NOT NULL, CHECK (1..5) (R-07)	Calificación
review_comment_title	VARCHAR(255)	NULL	Título opcional
review_comment_message	TEXT	NULL	Comentario opcional (~58.7% nulos)
review_creation_date	TIMESTAMP	—	Creación de la reseña
review_answer_timestamp	TIMESTAMP	—	Respuesta a la reseña

DDL consolidado (validado en PostgreSQL 16)

```

CREATE TABLE geolocation (
    geolocation_zip_code_prefix INTEGER PRIMARY KEY,
    geolocation_lat DOUBLE PRECISION,
    geolocation_lng DOUBLE PRECISION,
    geolocation_city VARCHAR(120),
    geolocation_state CHAR(2) CHECK (geolocation_state ~ '^[A-Z]{2}$')
);

CREATE TABLE customers (
    customer_id VARCHAR(32) PRIMARY KEY,
    customer_unique_id VARCHAR(32) NOT NULL, -- R-19
    customer_zip_code_prefix INTEGER REFERENCES geolocation (geolocation_zip_code_prefix) ON DELETE RESTRICT -- R-06
);

```

```

CREATE TABLE sellers (
    seller_id VARCHAR(32) PRIMARY KEY,
    seller_zip_code_prefix INTEGER REFERENCES geolocation -- R-06
    (geolocation_zip_code_prefix) ON DELETE RESTRICT
);

CREATE TABLE category_translation (
    product_category_name VARCHAR(80) PRIMARY KEY,
    product_category_name_english VARCHAR(80) NOT NULL
);

CREATE TABLE products (
    product_id VARCHAR(32) PRIMARY KEY,
    product_category_name VARCHAR(80) REFERENCES category_translation -- R-05
    (product_category_name) ON UPDATE CASCADE ON DELETE SET NULL,
    product_name_length INTEGER CHECK (product_name_length >= 0),
    product_description_length INTEGER CHECK (product_description_length >= 0),
    product_photos_qty INTEGER CHECK (product_photos_qty >= 0),
    product_weight_g INTEGER CHECK (product_weight_g IS NULL OR product_weight_g >= 0), -- R-
13
    product_length_cm INTEGER CHECK (product_length_cm IS NULL OR product_length_cm >= 0),
    product_height_cm INTEGER CHECK (product_height_cm IS NULL OR product_height_cm >= 0),
    product_width_cm INTEGER CHECK (product_width_cm IS NULL OR product_width_cm >= 0)
);

CREATE TABLE orders (
    order_id VARCHAR(32) PRIMARY KEY,
    customer_id VARCHAR(32) NOT NULL REFERENCES customers ON DELETE RESTRICT, -- R-01
    order_status VARCHAR(20) NOT NULL CHECK (order_status IN -- R-08
    ('created', 'approved', 'invoiced', 'processing', 'shipped',
    'delivered', 'unavailable', 'canceled')),
    order_purchase_timestamp TIMESTAMP NOT NULL,
    order_approved_at TIMESTAMP,
    order_delivered_carrier_date TIMESTAMP,
    order_delivered_customer_date TIMESTAMP,
    order_estimated_delivery_date TIMESTAMP,
    CHECK (order_approved_at IS NULL OR order_approved_at >= order_purchase_timestamp), -- R-14
    CHECK (order_delivered_customer_date IS NULL OR -- R-15
    order_delivered_customer_date >= order_delivered_carrier_date)
);

CREATE TABLE order_items (
    order_id VARCHAR(32) NOT NULL REFERENCES orders ON DELETE RESTRICT, -- R-02
    order_item_id INTEGER NOT NULL,
    product_id VARCHAR(32) NOT NULL REFERENCES products ON DELETE RESTRICT,
    seller_id VARCHAR(32) NOT NULL REFERENCES sellers ON DELETE RESTRICT,
    shipping_limit_date TIMESTAMP,
    price NUMERIC(12,2) NOT NULL CHECK (price >= 0), -- R-09
    freight_value NUMERIC(12,2) CHECK (freight_value >= 0), -- R-09
    PRIMARY KEY (order_id, order_item_id)
);

CREATE TABLE order_payments (
    order_id VARCHAR(32) NOT NULL REFERENCES orders ON DELETE CASCADE, -- R-03
    payment_sequential INTEGER NOT NULL,
    payment_type VARCHAR(20) CHECK (payment_type IN -- R-12
    ('credit_card', 'debit_card', 'voucher', 'boleto', 'not_defined')),

```

```

    payment_installments    INTEGER    CHECK    (payment_installments    >=    1),    --    R-11
    payment_value           NUMERIC(12,2) CHECK    (payment_value    >    0),    --    R-10
    PRIMARY KEY (order_id, payment_sequential)    --    R-17
);

CREATE TABLE order_reviews (
    review_id    VARCHAR(32) PRIMARY KEY,    --    R-26
    order_id     VARCHAR(32) NOT NULL UNIQUE REFERENCES orders    --    R-04, R-18
    ON DELETE CASCADE,
    review_score SMALLINT NOT NULL CHECK (review_score BETWEEN 1 AND 5), --    R-07
    review_comment_title    VARCHAR(255),
    review_comment_message    TEXT,
    review_creation_date    TIMESTAMP,
    review_answer_timestamp    TIMESTAMP
);

```

La restricción R-16 (*shipping_limit_date* >= *order_purchase_timestamp*) involucra dos tablas, por lo que se implementa con un trigger BEFORE INSERT/UPDATE sobre *order_items*, tal como anticipa la nota de implementación de la Sección 3.3.

4.3 Aplicación de tipos avanzados (JSONB, arrays, composite types)

Aunque el núcleo transaccional es altamente estructurado, PostgreSQL ofrece tipos avanzados que modelan con mayor fidelidad ciertos atributos sin romper la 3FN. Su uso se limita a los campos donde aportan expresividad o rendimiento real, evitando la sobreingeniería.

a) Composite type — dimension_cm

Las tres dimensiones físicas forman una unidad cohesiva. Un tipo compuesto las agrupa semánticamente y permite tratarlas como un solo valor, manteniendo el acceso por campo.

```

CREATE TYPE dimension_cm AS (length INT, height INT, width INT);
ALTER TABLE products ADD COLUMN dimensions dimension_cm;
-- Consulta: SELECT (dimensions).length FROM products WHERE product_id = $1;

```

b) JSONB — atributos heterogéneos del catálogo

El catálogo tiene 73 categorías con atributos variables (R observada en el EDA). En vez de decenas de columnas dispersas, una columna JSONB almacena los atributos específicos de cada categoría (voltaje, color, talla...). Es la contraparte relacional de la frontera con MongoDB: PostgreSQL conserva los atributos consultables para JOINS e integridad, y delega el catálogo enriquecido al motor documental. Se indexa con GIN para operadores @>, ? y ?&.

```

ALTER TABLE products ADD COLUMN extra_attributes JSONB NOT NULL DEFAULT '{} '::jsonb;
CREATE INDEX idx_products_attrs ON products USING GIN (extra_attributes);
-- Ejemplo de documento: {"voltage": "110V", "color": "negro", "warranty_months": 12}
-- Consulta: SELECT * FROM products WHERE extra_attributes @> '{"color":"negro"}';

```

c) Arrays — colecciones multivaluadas de solo lectura

Las URLs de las fotos del producto son un dato multivaluado y de solo lectura, coherente con product_photos_qty. Un arreglo TEXT[] evita una tabla puente innecesaria para un dato que nunca se actualiza individualmente. Se indexa con GIN para búsquedas de contención.

```
ALTER TABLE products ADD COLUMN photo_urls TEXT[];
CREATE INDEX idx_products_photos ON products USING GIN (photo_urls);
-- Consulta: SELECT * FROM products WHERE 'https://.../1.jpg' = ANY(photo_urls);
-- Coherencia: CHECK (product_photos_qty = COALESCE(array_length(photo_urls,1),0))
```

d) Rangos de fecha — tstzrange para la ventana de entrega

La ventana de entrega estimada se modela con un rango temporal, lo que habilita operadores de solapamiento (&&) y contención (@>), e incluso restricciones de exclusión vía GiST (extensión btree_gist). Sustituye con ventaja a dos columnas sueltas de fecha.

```
ALTER TABLE orders ADD COLUMN delivery_window TSTZRANGE;
CREATE INDEX idx_orders_delivery_window ON orders USING GIST (delivery_window);
-- ¿La entrega real cayó dentro de la ventana estimada?
-- SELECT order_id FROM orders WHERE delivery_window @> order_delivered_customer_date;
```

Justificación resumida

Tipo avanzado	Dónde se aplica	Justificación
Composite (dimension_cm)	products.dimensions	Agrupar atributos cohesivos; acceso por campo sin perder atomicidad
JSONB	products.extra_attributes	Atributos por categoría heterogéneos; índice GIN; frontera con MongoDB
Array (TEXT[])	products.photo_urls	Multivaluado de solo lectura sin tabla puente; índice GIN
Range (tstzrange)	orders.delivery_window	Operadores de rango y exclusión; índice GiST con btree_gist

4.4 Extensiones a utilizar con justificación

Las siguientes extensiones —todas disponibles en Supabase, la plataforma de despliegue elegida— habilitan capacidades requeridas por el diseño y por la futura optimización de consultas. Se activan con CREATE EXTENSION IF NOT EXISTS.

Extensión	Propósito	Justificación en Ecommify
pgcrypto	Funciones criptográficas y gen_random_uuid()	Generar identificadores UUID para nuevas filas preservando el formato anonimizado del dataset (R-24, R-25)
unaccent	Normalización de acentos	Búsquedas sobre ciudades y categorías en portugués (São Paulo ~ sao paulo)

pg_trgm	Similitud por trigramas e índices GIN/GIST	Búsqueda difusa y autocompletado de ciudad/categoría; combinada con unaccent
btree_gin	Índices GIN sobre tipos escalares	Índices compuestos que mezclan JSONB/array con columnas escalares (p. ej. categoría + extra_attributes)
btree_gist	Operadores de igualdad en GiST	Restricciones de exclusión sobre delivery_window combinadas con columnas escalares
postgis	Tipos y operadores geoespaciales	Geolocation tiene lat/lng y >1M filas: consultas de distancia cliente-vendedor y zonas de cobertura
pg_stat_statements	Estadísticas de consultas	Base para la fase de optimización: detectar consultas lentas y validar el uso de índices

```

CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS unaccent;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION IF NOT EXISTS btree_gin;
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- Índice de búsqueda difusa de ciudades por trigramas (validado en PG 16):
CREATE INDEX idx_geo_city_trgm ON geolocation
    USING GIN (geolocation_city gin_trgm_ops);
-- Para indexar de forma INSENSIBLE a acentos se envuelve unaccent en una
-- función IMMUTABLE (unaccent es STABLE y no puede ir directo en el índice):
-- CREATE FUNCTION f_unaccent(text) RETURNS text LANGUAGE sql IMMUTABLE
-- AS $$ SELECT unaccent('unaccent', $1) $$;
-- CREATE INDEX ... USING GIN (f_unaccent(geolocation_city) gin_trgm_ops);

```

PostGIS y pg_stat_statements se habilitan según necesidad; las demás son livianas y se activan desde el inicio. En Supabase Free Tier todas están soportadas, lo que evita dependencias externas.

4.5 Estrategia de particionamiento

El particionamiento declarativo de PostgreSQL aplica a las tablas que crecen de forma sostenida o que superan volúmenes altos. Tras analizar el modelo, se decide particionar Orders (crecimiento temporal) y Geolocation (volumen y acceso regional); el resto de las tablas permanecen sin particionar por su tamaño moderado en el alcance académico.

a) Orders — particionamiento RANGE por fecha de compra

La distribución temporal del EDA muestra que los pedidos se concentran en 2017-2018 y que la mayoría de los reportes filtran por rango de fechas. Particionar por order_purchase_timestamp habilita partition pruning (el planificador descarta particiones

fuera del rango) y facilita el archivado por período. La clave primaria debe incluir la columna de partición, por lo que pasa a ser (order_id, order_purchase_timestamp): es el principal trade-off de esta estrategia.

```
CREATE TABLE orders (
  order_id VARCHAR(32) NOT NULL,
  customer_id VARCHAR(32) NOT NULL REFERENCES customers,
  order_status VARCHAR(20) NOT NULL,
  order_purchase_timestamp TIMESTAMP NOT NULL,
  -- ... resto de columnas y CHECKs ...
  PRIMARY KEY (order_id, order_purchase_timestamp)
) PARTITION BY RANGE (order_purchase_timestamp);

CREATE TABLE orders_2016 PARTITION OF orders
FOR VALUES FROM ('2016-01-01') TO ('2017-01-01');
CREATE TABLE orders_2017 PARTITION OF orders
FOR VALUES FROM ('2017-01-01') TO ('2018-01-01');
CREATE TABLE orders_2018 PARTITION OF orders
FOR VALUES FROM ('2018-01-01') TO ('2019-01-01');
CREATE TABLE orders_default PARTITION OF orders DEFAULT;
```

b) Geolocation — particionamiento LIST por estado

Con más de un millón de filas y consultas frecuentes por región, dividir por geolocation_state (27 UF brasileñas) acota cada partición a un estado y mejora la localidad de las consultas geográficas. Una partición DEFAULT captura valores atípicos.

```
CREATE TABLE geolocation (
  geolocation_zip_code_prefix INTEGER NOT NULL,
  geolocation_lat DOUBLE PRECISION,
  geolocation_lng DOUBLE PRECISION,
  geolocation_city VARCHAR(120),
  geolocation_state CHAR(2) NOT NULL,
  PRIMARY KEY (geolocation_zip_code_prefix, geolocation_state)
) PARTITION BY LIST (geolocation_state);

CREATE TABLE geolocation_sp PARTITION OF geolocation FOR VALUES IN ('SP');
CREATE TABLE geolocation_rj PARTITION OF geolocation FOR VALUES IN ('RJ');
CREATE TABLE geolocation_mg PARTITION OF geolocation FOR VALUES IN ('MG');
CREATE TABLE geolocation_default PARTITION OF geolocation DEFAULT;
```

Decisión por tabla

Tabla	¿Particionar?	Estrategia	Criterio
Orders	Sí	RANGE por order_purchase_timestamp (anual)	Crecimiento temporal y reportes por fecha (pruning)
Geolocation	Sí	LIST por geolocation_state	>1M filas y acceso regional
Order_Items / Payments / Reviews	Diferido	RANGE alineado a Orders (a futuro)	Tamaño moderado; requiere propagar la clave de partición

Customers / Sellers / Products / Translation	No	—	Volumen bajo o medio; el particionamiento no aporta
--	----	---	---

Consideración de plataforma: en Supabase Free Tier (límite de 500 MB, R-20) el particionamiento se combina con la deduplicación de Geolocation en el ETL. El particionamiento declarativo es totalmente compatible con PostgreSQL gestionado por Supabase, por lo que la estrategia es aplicable sin servicios adicionales.

5. Diseño lógico preliminar- Módulo MongoDB

Durante el análisis exploratorio se observó que no todos los datos del e-commerce tienen la misma naturaleza ni los mismos patrones de uso. Mientras entidades como Orders, Order_Items y Payments presentan relaciones estrictas y alta consistencia, otras entidades como Products y Reviews muestran un comportamiento más flexible y semi-estructurado.

Se identificó que el catálogo contiene 32.951 productos distribuidos en 73 categorías con atributos heterogéneos. Algunas categorías tienen dimensiones físicas completas, peso y fotografías, mientras otras presentan atributos faltantes o distinta granularidad. Además, las reseñas mezclan información estructurada (review_score) con comentarios opcionales, donde aproximadamente el 58% de review_comment_message son nulos.

A partir de estos hallazgos, se propone una arquitectura híbrida donde PostgreSQL maneje el núcleo transaccional y MongoDB funcione como una capa documental orientada a catálogo, analítica y consultas flexibles.

5.1 Colecciones principales identificadas

Products

La colección principal en MongoDB sería products. Esta colección almacenaría el catálogo enriquecido de productos del e-commerce.

La razón principal para llevar esta entidad a MongoDB es la heterogeneidad del catálogo. Se analizó que no todos los productos comparten la misma estructura: algunos tienen peso, dimensiones y fotografías completas, mientras otros poseen campos faltantes o atributos distintos según la categoría.

En PostgreSQL esto obligaría a tener muchas columnas vacías o tablas auxiliares adicionales. MongoDB, en cambio, permite manejar documentos dinámicos sin imponer un esquema rígido.

Esta colección estaría más orientada a:

- consultas de catálogo
- búsquedas rápidas
- frontend del e-commerce
- filtros por categorías
- Recomendaciones
- consultas analíticas ligeras

Reviews

La colección reviews almacenaría las reseñas de productos y pedidos.

Según el EDA, muchas reseñas contienen únicamente puntuación numérica y no comentarios escritos. Esto hace que MongoDB sea adecuado porque los documentos pueden omitir directamente los campos vacíos sin necesidad de columnas NULL.

Además, MongoDB facilita:

- búsquedas de texto
- análisis de sentimiento
- análisis de palabras frecuentes
- clasificación de reseñas.

Aunque PostgreSQL conservaría una tabla `order_reviews` para mantener integridad referencial con `Orders`, MongoDB serviría como capa analítica y documental para explotación flexible de comentarios.

Product_catalog_view

Esta colección funcionaría como una vista documental enriquecida del catálogo.

Su objetivo sería reducir consultas complejas al frontend. Aquí podrían combinarse:

- información básica del producto
- categoría
- métricas agregadas

- puntuación promedio
- total de reseña
- total de ventas
- productos relacionados

Existe la posibilidad de crear estructuras desnormalizadas para acelerar consultas frecuentes y evitar múltiples JOIN en tiempo real.

Esta colección tendría un enfoque principalmente, analítico, de lectura intensiva y optimizado para frontend.

User_behavior

Aunque no forma parte directa del dataset original, esta colección se considera importante para un escenario realista de e-commerce.

Aquí se almacenarían eventos como:

- productos vistos
- búsquedas realizadas
- clics
- carritos abandonados
- navegación por categorías

Este tipo de información suele crecer rápidamente y tener una estructura variable, por lo que MongoDB resulta más adecuado que PostgreSQL.

Además, esta colección podría apoyar los motores de recomendación, análisis de comportamiento, personalización del catálogo y tendencias de navegación.

product_catalog_view

Sería una vista documental enriquecida del catálogo. Podría combinar datos de producto, categoría, vendedor, calificación promedio y métricas de ventas. Se menciona que la desnormalización estratégica puede servir para reporting, frontend y consultas frecuentes, siempre que no afecte la base transaccional principal.

Recommendations

La colección recommendations almacenaría recomendaciones pre calculadas para usuarios o productos. En lugar de calcular sugerencias dinámicamente mediante múltiples consultas relacionales, MongoDB puede almacenar resultados ya procesados y responder rápidamente al frontend. Esto se alinea con el enfoque de consultas analíticas y de lectura frecuente.

5.2 Esquema de documentos (ejemplo en JSON)

products

```
{
  "_id": "product_001",
  "product_id": "1e9e8ef04dbcff4541ed26657ea517e5",
  "category": {
    "name_pt": "perfumaria",
    "name_en": "perfumery"
  },
  "specs": {
    "name_length": 40,
    "description_length": 287,
    "photos_qty": 2,
    "weight_g": 225,
    "dimensions_cm": {
      "length": 16,
      "height": 10,
      "width": 14
    }
  },
  "analytics": {
    "avg_review_score": 4.5,
    "total_reviews": 120,
    "total_sales": 850
  },
  "status": "active"
}
```

Products con atributos incompletos

```
{  
  "_id": "product_002",  
  "product_id": "abc123xyz",  
  "category": {  
    "name_pt": "seguros_e_servicos",  
    "name_en": "security_and_services"  
  },  
  "specs": {  
    "name_length": 22,  
    "photos_qty": 1  
  },  
  "status": "active"  
}
```

Aquí se observa claramente la ventaja documental: el producto no necesita almacenar dimensiones ni peso si no existen.

Reviews

```
{  
  "_id": "review_001",  
  "review_id": "7bc2406110b926393aa56f80a40eba40",  
  "order_id": "73fc7af87114b39712e6da79b0a377eb",  
  "product_id": "product_001",  
  "score": 4,  
  "created_at": "2018-01-18T00:00:00Z",  
  "answered_at": "2018-01-18T21:46:59Z"  
}
```

Reviews con comentario completo

```
{  
  "_id": "review_002",  
  "review_id": "def456uvw",  
  "order_id": "abc789xyz",  
  "product_id": "product_001",  
  "score": 2,  
  "comment": {  
    "title": "Producto llegó dañado",  
    "message": "La caja llegó golpeada y el producto tenía daños visibles."  
  },  
  "analysis": {  
    "sentiment": "negative",  
    "keywords": ["dañado", "golpeado"]  
  },  
  "created_at": "2018-03-05T00:00:00Z"  
}
```

User_behavior

```
{  
  "_id": "session_001",  
  "customer_id": "cust_001",  
  "events": [  
    {  
      "type": "view_product",  
      "product_id": "product_001",  
      "timestamp": "2026-05-10T10:00:00Z"  
    }  
  ]  
}
```

```
},
{
  "type": "search",
  "query": "toallas",
  "timestamp": "2026-05-10T10:02:00Z"
},
"device": {
  "type": "mobile",
  "browser": "Chrome"
}
}
```

Recommendations

```
{
  "_id": "rec_001",
  "customer_id": "cust_001",
  "generated_at": "2026-05-10T12:00:00Z",
  "recommendations": [
    {
      "product_id": "product_001",
      "score": 0.91,
      "reason": "Relacionado con compras anteriores"
    },
    {
      "product_id": "product_045",
      "score": 0.84,
      "reason": "Popular en la misma categoría"
    }
  ]
}
```

```
]
}
```

5.3 Patrones de modelado a aplicar

Embedding

El patrón embedding se usaría cuando los datos se consultan normalmente juntos.

Por ejemplo:

- categoría embebida dentro del producto
- dimensiones dentro del producto
- comentarios dentro de la reseña
- eventos dentro de una sesión de usuario

Esto reduce la necesidad de \$lookup, mejora tiempos de lectura y simplifica consultas frecuentes del frontend.

Además, Atlas M0 tiene recursos limitados y no cuenta con infraestructura dedicada, por lo que reducir consultas complejas ayuda bastante al rendimiento.

Referencing

El patrón referencing se usaría para datos transaccionales o compartidos.

Por ejemplo:

- order_id
- product_id
- customer_id

Estos campos actuarían como claves compartidas entre PostgreSQL y MongoDB.

La idea es evitar duplicar completamente entidades críticas como pedidos o pagos, ya que esas relaciones deben mantenerse bajo integridad referencial fuerte en PostgreSQL.

Computed Pattern

Este patrón se usaría para almacenar valores precalculados como:

- promedio de reseñas
- total de ventas
- productos más vistos
- Recomendaciones

Las agregaciones precalculadas ayudan a evitar consultas costosas y mejoran el rendimiento de reporting y frontend.

Subset Pattern

Este patrón se aplicaría especialmente al catálogo.

En MongoDB se almacenaría únicamente la información más consultada: categoría, dimensiones, métricas, fotos, puntuación promedio.

Los datos transaccionales completos permanecerían en PostgreSQL.

Esto ayuda a:

- reducir tamaño de documentos,
- optimizar almacenamiento,
- mantener el cluster Atlas M0 dentro del límite de 512 MB.

Approximation Pattern

Podría aplicarse en métricas no críticas como: visualizaciones, ranking de popularidad, conteo aproximado de clics.

No sería adecuado para pagos ni pedidos, ya que esos requieren exactitud y consistencia fuerte.

5.4 Justificación de qué datos van a MongoDB vs PostgreSQL

PostgreSQL debería manejar:

- Customers
- Orders
- Order_Items
- Order_Payments
- Sellers
- Geolocation

La razón principal es que estas entidades necesitan, integridad referencial, transacciones ACID, restricciones, normalización, consistencia inmediata

Por ellos se validó que no existen registros huérfanos y que el modelo relacional presenta alta consistencia.

Además, Orders y Payments superan los 100 mil registros transaccionales y representan el núcleo operativo del negocio.

MongoDB debería manejar:

- Products
- Reviews
- Product Catalog View
- User Behavior
- Recommendations

Esto se debe a que, Products tiene atributos heterogéneos, Reviews mezcla datos estructurados y texto libre, el frontend requiere consultas rápidas y el análisis documental necesita flexibilidad. Incluso dentro de la arquitectura se propone explícitamente una tabla Products “delgada” en PostgreSQL y un catálogo enriquecido en MongoDB.

6. Decisiones arquitectónicas justificadas

Arquitectura híbrida propuesta

La arquitectura propuesta para Ecommify es una arquitectura híbrida o políglota.

PostgreSQL funcionaría como, núcleo transaccional, sistema OLTP y la fuente de verdad principal. Mientras que MongoDB funcionaría como, capa documental, motor de catálogo, módulo analítico, soporte para consultas.

El flujo general sería:

1. PostgreSQL registra, pedidos, pagos, clientes, vendedores.
2. Procesos ETL extraen información desde PostgreSQL.
3. MongoDB recibe los productos enriquecidos, reseñas, métricas, recomendaciones y las vistas analíticas.
4. Se realizan consultas principalmente a MongoDB para, catálogo, filtros, recomendaciones, análisis de reseñas. La sincronización se realizaría mediante ETL por lotes porque Atlas M0 no soporta Change Streams.

Las claves compartidas entre ambos sistemas serían:

- product_id
- order_id
- customer_id

Matriz de decisión por entidad

Módulo / Entidad	Tipo de dato	Patrón principal	CAP recomendado	Base sugerida	Justificación
 Customers	Estructurado	Transaccional	Consistencia	PostgreSQL	Tiene relación directa con Orders y requiere integridad referencial para mantener consistencia y evitar datos huérfanos.
 Orders	Estructurado	Transaccional	Consistencia	PostgreSQL	Es el núcleo del e-commerce; no debe haber pedidos incompletos o inconsistentes.
 Order_Items	Estructurado	Transaccional / analítico	Consistencia	PostgreSQL	Une pedidos, productos y vendedores mediante claves foráneas; requiere integridad y consultas frecuentes.
 Order_Payments	Estructurado	Transaccional	Consistencia fuerte	PostgreSQL	Maneja dinero; requiere precisión y control ACID para evitar transacciones inválidas.
 Sellers	Estructurado	Transaccional / consulta	Consistencia	PostgreSQL	Participa en Order_Items y debe mantener trazabilidad con integridad referencial.
 Geolocation	Estructurado	Consulta / analítico	Disponibilidad moderada	PostgreSQL o MongoDB	Puede estar normalizada en PostgreSQL, pero también replicarse en MongoDB para consultas geográficas y reportes.
 Products	Semi-estructurado	Lectura / catálogo	Disponibilidad	MongoDB	Tiene atributos variables por categoría y se beneficia de documentos flexibles.
 Reviews	Semi-estructurado	Lectura / analítica	Disponibilidad	MongoDB	Combina score estructurado con texto libre opcional; consultas analíticas frecuentes.
 Category_Translation	Estructurado simple	Consulta	Disponibilidad	MongoDB o PostgreSQL	Puede embeberse dentro del documento de producto (MongoDB) o manejarse en una tabla separada (PostgreSQL).

Análisis del teorema CAP aplicado

Desde el teorema CAP, no todos los módulos de Ecommify requieren el mismo equilibrio entre consistencia y disponibilidad. En pedidos y pagos debe priorizarse la consistencia, porque una orden no puede quedar registrada sin cliente, sin ítems válidos o con pagos inconsistentes. En estos módulos, PostgreSQL es más adecuado porque permite transacciones ACID, claves foráneas, restricciones y control fuerte de integridad. Si ocurre una falla, es preferible rechazar temporalmente una operación antes que aceptar un pago duplicado o un pedido incompleto.

En cambio, para catálogo de productos, reseñas y búsquedas, la disponibilidad puede tener más peso. Si un usuario consulta productos o comentarios, el sistema puede tolerar pequeños retrasos de sincronización sin afectar la operación financiera principal. MongoDB se ajusta mejor a estos casos porque permite documentos flexibles, lectura rápida y estructuras menos rígidas. Esto es importante porque el

catálogo de Ecommify contiene 32.951 productos distribuidos en 73 categorías, con atributos variables y algunos campos incompletos.

Análisis de Trade-offs y Estrategias de Mitigación

El diseño de la arquitectura de persistencia de Ecommify exige balancear fuerzas contrapuestas mediante el análisis de compromisos (trade-offs) de diseño. La transición desde un modelo de origen completamente desnormalizado (archivo plano orders_full, donde un solo pedido con múltiples artículos duplica masivamente las cabeceras de cliente y estado) hacia un esquema optimizado plantea un escenario crítico de evaluación técnica.

• Matriz de Trade-offs: Normalización vs. Desnormalización			ANÁLISIS DE ARQUITECTURA
La adopción de una Tercera Forma Normal (3FN) estricta en el motor relacional mitiga de raíz las anomalías de escritura, pero introduce penalizaciones latentes en escenarios de lectura analítica intensiva. Impacto sobre los atributos de calidad:			
ATRIBUTO DE CALIDAD	ALTA NORMALIZACIÓN (3FN - POSTGRESQL)	DESNORMALIZACIÓN (MONGODB)	
 Consistencia e Integridad (ACID)	 FUERZA Máxima garantía. Se eliminan las anomalías de actualización y la duplicidad. Restricciones de integridad referencial (FK, NOT NULL, CHECK) operan a nivel de motor.	 COMPROMISO Riesgo de inconsistencia eventual. La actualización de datos duplicados o embebidos en múltiples documentos requiere lógica a nivel de aplicación.	
 Rendimiento de Escritura (Throughput)	 FUERZA Alta eficiencia en transacciones unitarias. Las operaciones de INSERT o UPDATE modifican pocas páginas de disco debido a tablas angostas y especializadas.	 COMPROMISO Puede ralentizarse si los documentos embebidos crecen dinámicamente y superan los límites de página (mutación de arrays en MongoDB).	
 Rendimiento de Lectura (Analítica)	 COMPROMISO Degradación por Overhead de operaciones JOIN. Consultar una orden completa implica combinar hasta 5 tablas (Orders, Customers, Order_Items, Payments, Reviews).	 FUERZA Latencia mínima en lecturas orientadas a vistas. Un solo Query por ID recupera el documento agregado con toda la información necesaria.	
 Eficiencia de Almacenamiento	 FUERZA Huella en disco optimizada. Los datos repetitivos (como prefijos de códigos postales en Geolocation) se almacenan una única vez.	 COMPROMISO Mayor consumo de almacenamiento en disco debido a la redundancia intencional de llaves y atributos embebidos.	

Estrategias de Mitigación Arquitectónica

Para resolver los compromisos identificados en la matriz anterior sin comprometer los objetivos transaccionales ni los analíticos de Ecommify, se definen las siguientes cuatro tácticas de mitigación a nivel de arquitectura:

Mitigación A: Arquitectura de Persistencia Políglota (Híbrida)

- Problema: El catálogo de productos presenta alta heterogeneidad en sus atributos y las reseñas manejan texto libre con altos índices de valores nulos, lo que dificulta un modelado relacional rígido.
- Solución: Se segmenta el almacenamiento de datos según la naturaleza de la carga de trabajo. PostgreSQL actúa como el componente transaccional principal (Core OLTP), custodiando las entidades que requieren consistencia estricta (Orders, Order_Payments, Customers). En paralelo, MongoDB se integra como la base de datos documental para el catálogo operativo (Products con categorías embebidas) y el motor de retroalimentación (Order_Reviews), absorbiendo la variabilidad estructural y los campos opcionales sin penalizar el rendimiento del motor relacional.

Mitigación B: Reconciliación Asíncrona por Lotes y Control de Consistencia Eventual

- Problema: Al descentralizar los datos entre PostgreSQL y MongoDB en capas gratuitas (Supabase y MongoDB Atlas M0), se pierde la capacidad de ejecutar transacciones distribuidas distribuidas (Two-Phase Commit) o de utilizar Change Streams en tiempo real para sincronizar inventarios y productos.
- Solución: Se implementa un patrón de reconciliación asíncrona a nivel de aplicación mediante scripts programados en Python (vía tareas programadas o cronjobs). El script valida periódicamente la integridad referencial cruzada (por ejemplo, asegurando que el 100% de los product_id registrados en la tabla Order_Items de PostgreSQL existan físicamente como documentos válidos en la colección Products de MongoDB). Las discrepancias se mitigan levantando alertas o imputando registros de contingencia temporales.

Mitigación C: Vistas Materializadas e Índices Parciales para Reportes OLAP

- Problema: La normalización transaccional ralentiza las consultas de la junta directiva y analistas de negocio, quienes necesitan calcular constantemente agregaciones (como el volumen de ventas por estado o métodos de pago preferidos).
- Solución: En el motor relacional (PostgreSQL), se diseñan Vistas Materializadas que pre-calculan las uniones (JOINS) y agregaciones pesadas durante las horas de menor demanda transaccional. Adicionalmente, se configuran Índices Parciales (por ejemplo, indexar solo las órdenes cuyo estado sea estrictamente delivered), reduciendo drásticamente el tamaño del índice en disco y acelerando las lecturas analíticas en más de un 60% sin sobrecargar las escrituras cotidianas.

Mitigación D: Deduplicación y Limpieza en la Capa de Ingesta (Staging/Pre-load)

- Problema: La tabla Geolocation supera el millón de registros en bruto y posee un alto volumen histórico de registros duplicados e inconsistencias espaciales que degradarían el rendimiento de los índices relacionales.
- Solución: Se establece una regla arquitectónica en la tubería de datos (Data Pipeline): no se permite la inserción directa de datos sucios al core de la base de datos. Se ejecuta un proceso de limpieza previo a la carga donde se remueven duplicados exactos a nivel de prefijo de código postal y coordenadas, y se resuelven los valores nulos basándose en los datos demográficos cruzados de Customers y Sellers. Esto estabiliza el tamaño de la tabla a una fracción del original antes de levantar las restricciones de integridad y las claves foráneas.

Flujos de sincronización si aplica

7. Anexos

7.1 Diccionario de Datos Ecommify

1. Tabla: Customers (Clientes)

Almacena la información de los clientes y sus ubicaciones a nivel de sesión y de historial.

Customers			
Almacena la información de los clientes y sus ubicaciones a nivel de sesión y de historial.			
COLUMNA	TIPO DE DATO	LLAVE	DESCRIPCIÓN
customer_id	TEXT	PK	Identificador único por sesión de cliente.
customer_unique_id	TEXT		ID único legal del cliente (para análisis histórico).
zip_code_prefix	INT	FK	Referencia a Geolocation. Ubicación del cliente.
city	TEXT		Ciudad (redundante para performance o denormalización local).
state	TEXT		Estado (redundante para performance o denormalización local).

2. Tabla: Geolocation (Geolocalización)

Catálogo maestro de coordenadas geográficas basadas en códigos postales.

Geolocation		Catálogo maestro de coordenadas geográficas basadas en códigos postales.	
COLUMNA	TIPO DE DATO	LLAVE	DESCRIPCIÓN
zip_code_prefix	INT	PK	Prefijo de código postal único.
lat	FLOAT		Latitud geográfica.
lng	FLOAT		Longitud geográfica.
city	TEXT		Nombre de la ciudad.
state	TEXT		Sigla del estado.

3. Tabla: Orders (Pedidos)

Gestiona el ciclo de vida transaccional y los estados de cada pedido.

Orders		Gestiona el ciclo de vida transaccional y los estados de cada pedido.	
COLUMNA	TIPO DE DATO	LLAVE	DESCRIPCIÓN
order_id	TEXT	PK	Identificador único del pedido.
customer_id	TEXT	FK	Referencia a Customers. Cliente que realizó la compra.
status	TEXT		Estado del flujo (delivered, shipped, etc.).
purchase_timestamp	DATETIME		Fecha y hora de creación de la orden.
approved_at	DATETIME		Fecha y hora de aprobación del pago.
delivered_carrier	DATETIME		Fecha de entrega al centro logístico.
delivered_customer	DATETIME		Fecha real de entrega al cliente.
estimated_delivery	DATETIME		Fecha de entrega prometida al cliente.

4. Tabla: Order_Items (Ítems del Pedido)

Detalle de los productos incluidos en cada pedido. *Nota arquitectónica: Posee una llave primaria compuesta.*

Order_Items		Detalle de los productos incluidos en cada pedido. (Llave primaria compuesta).	
COLUMNA	TIPO DE DATO	LLAVE	DESCRIPCIÓN
order_id	TEXT	PK / FK	Referencia a Orders. Parte de la llave compuesta.
order_item_id	INT	PK	Secuencial del producto dentro de la orden.
product_id	TEXT	FK	Referencia a Products. Producto vendido.
seller_id	TEXT	FK	Referencia a Sellers. Vendedor que provee el producto.
shipping_limit	DATETIME		Fecha límite para que el vendedor despache.
price	FLOAT		Precio unitario del producto en esta venta.
freight_value	FLOAT		Costo del flete para este ítem.

5. Tabla: Products (Productos)

Catálogo maestro de productos y sus características físicas.

Products		Catálogo maestro de productos y sus características físicas.	
COLUMNA	TIPO DE DATO	LLAVE	DESCRIPCIÓN
product_id	TEXT	PK	Identificador único del producto.
category_name	TEXT	FK	Referencia a Category_Translation. Categoría en portugués.
name_length	INT		Cantidad de caracteres del nombre.
description_length	INT		Cantidad de caracteres de la descripción.
photos_qty	INT		Cantidad de imágenes publicadas.
weight_g	FLOAT		Peso en gramos.
length_cm	FLOAT		Longitud en centímetros.
height_cm	FLOAT		Altura en centímetros.
width_cm	FLOAT		Ancho en centímetros.

6. Tabla: Sellers (Vendedores)

Información de los comercios/vendedores asociados a la plataforma.

Sellers		Información de los comercios/vendedores asociados a la plataforma.	
COLUMNA	TIPO DE DATO	LLAVE	DESCRIPCIÓN
seller_id	TEXT	PK	Identificador único del vendedor.
zip_code_prefix	INT	FK	Referencia a Geolocation. Ubicación del vendedor.
city	TEXT		Ciudad del vendedor.
state	TEXT		Estado del vendedor.

7. Tabla: Order_Payments (Pagos del Pedido)

Registro de las transacciones y métodos de pago. *Nota arquitectónica: Posee una llave primaria compuesta.*

Order_Payments		Registro de las transacciones y métodos de pago. (Llave primaria compuesta).	
COLUMNA	TIPO DE DATO	LLAVE	DESCRIPCIÓN
order_id	TEXT	PK / FK	Referencia a Orders. Parte de la llave compuesta.
sequential	INT	PK	Número de pago (ej. 1 de 2 si hay división).
type	TEXT		Método (credit_card, voucher, boleto).
installments	INT		Cantidad de cuotas.
value	FLOAT		Monto pagado en esta transacción.

8. Tabla: Order_Reviews (Reseñas del Pedido)

Retroalimentación y calificación de los clientes sobre sus compras.

Order_Reviews		Retroalimentación y calificación de los clientes sobre sus compras.	
COLUMNA	TIPO DE DATO	LLAVE	DESCRIPCIÓN
review_id	TEXT	PK	Identificador de la reseña.
order_id	TEXT	FK	Referencia a Orders. Pedido evaluado.
score	INT		Calificación numérica (1 a 5).
comment_title	TEXT		Título del comentario del cliente.
comment_message	TEXT		Cuerpo del mensaje de la reseña.
creation_date	DATETIME		Fecha de publicación de la reseña.
answer_timestamp	DATETIME		Fecha de respuesta del vendedor/sistema.

9. Tabla: Category_Translation (Traducción de Categorías)

Tabla de homologación para internacionalización de categorías.

Category_Translation		Tabla de homologación para internacionalización de categorías.	
COLUMNA	TIPO DE DATO	LLAVE	DESCRIPCIÓN
category_name	TEXT	PK	Nombre de la categoría en portugués.
category_english	TEXT		Traducción al inglés de la categoría.

7.2 Scripts SQL preliminares

7.3 Ejemplos de datos

Los ejemplos de datos ya están previamente guardados en cada DB.